

Technical Interview Coach

Prepared By: Bhargav Doddi

Role: AI Automation Engineer

Project Category: Conversational AI

Assessment Framework: GenAI Assessment Framework 2026

Date: July 2026

Document Information

Item	Details
Project Name	Technical Interview Coach
Document Type	Software Design Document
Version	1.0
Author	Bhargav Doddi
Frontend	React.js
Backend	FastAPI
Programming Language	Python 3.12+
AI Framework	LangChain

AI Model	Google Gemini
----------	---------------

Database	SQLite
----------	--------

Communication	REST API
---------------	----------

Revision History

Version	Date	Description	Author
1.0	July 2026	Initial Release	Bhargav Doddi

Executive Summary

The **Technical Interview Coach** is an AI-powered web application designed to help students and professionals prepare for technical interviews through interactive conversations. Instead of practising with static questionnaires, users participate in realistic interview sessions conducted by an AI interviewer.

The system generates interview questions based on the selected technology and difficulty level, analyses user responses, provides personalised feedback, and stores interview history for future review. The application combines modern web technologies with Large Language Models (LLMs) to create an engaging and effective learning experience.

The application is built using **React.js** for the frontend, **FastAPI** for backend services, **Google Gemini** for AI-powered conversations, **LangChain** for prompt orchestration, and **SQLite** for data storage. The modular architecture allows future integration of additional interview domains, coding assessments, voice interviews, and cloud-based deployment.

This Software Design Document describes the architecture, modules, database design, APIs, security mechanisms, deployment strategy, and testing approach adopted during the development of the Technical Interview Coach.

Table of Contents

1. Introduction
2. Overall System Description
3. Software Architecture
4. Technology Stack
5. Database Design
6. API Design
7. User Interface Design
8. Testing Strategy
9. Deployment and Security
10. Future Enhancements and Conclusion
11. References

Chapter 1 – Introduction

1.1 Overview

Technical interviews are one of the most challenging stages of the recruitment process. Many candidates struggle because they have limited opportunities to practise realistic interview scenarios. The Technical Interview Coach addresses this challenge by providing an AI-powered platform that simulates technical interviews across multiple domains.

The system interacts with users through conversational AI, generates relevant technical questions, evaluates responses, and delivers constructive feedback. Unlike traditional practice platforms, the application adapts questions according to the selected technology and difficulty level, making every interview session more personalised and practical.

1.2 Objectives

The primary objectives of the Technical Interview Coach are:

- Provide realistic AI-driven mock interviews.
- Generate technical questions dynamically.
- Evaluate candidate responses intelligently.
- Deliver personalised feedback and improvement suggestions.
- Maintain interview history for progress tracking.
- Improve interview confidence through repeated practice.

1.3 Scope

The current version of the application focuses on text-based technical interviews. Users can choose an interview domain, answer AI-generated questions, and receive automated evaluations.

The project currently supports:

- Python
- Java
- JavaScript
- SQL
- Data Structures
- Operating Systems
- Computer Networks
- System Design

The application does not currently include voice interviews, video interviews, live coding assessments, or recruiter dashboards. These capabilities are considered future enhancements.

1.4 Target Users

User Category	Purpose
Students	Campus interview preparation
Fresh Graduates	Entry-level technical interviews
Software Engineers	Technical skill revision
Career Switchers	Upskilling before interviews

1.5 Expected Benefits

The application offers several benefits to its users:

- Practice interviews at any time.
- Receive instant AI-generated feedback.
- Improve technical communication skills.
- Identify strengths and weak areas.
- Build confidence before actual interviews.

1.6 Assumptions

The system assumes that users have:

- Basic knowledge of programming.
- Internet connectivity.
- A modern web browser.
- Access to a valid Gemini API key through the backend configuration.

1.7 Constraints

The project operates under the following constraints:

- Internet connection is required for AI responses.
- AI evaluation quality depends on the LLM.
- SQLite limits concurrent users.
- Voice interaction is not supported in the current release.

1.8 Chapter Summary

This chapter introduced the Technical Interview Coach by describing its purpose, objectives, scope, intended users, expected benefits, assumptions, and project limitations. The following chapter explains the overall system requirements and functional behaviour.

Chapter 2 – Overall System Description

2.1 System Overview

The Technical Interview Coach is a web-based Conversational AI application that helps users prepare for technical interviews through interactive AI-driven conversations. The system follows a client-server architecture where the React frontend communicates with the FastAPI backend using REST APIs. The backend processes user requests, interacts with the Google Gemini model to generate interview questions and feedback, and stores interview records in a SQLite database.

2.2 System Components

Component	Description
React.js	Provides a responsive user interface for interview sessions.
FastAPI	Implements business logic and REST API services.

Google Gemini	Generates interview questions and evaluates responses.
LangChain	Manages prompt execution and AI interaction.
SQLite	Stores interview history, responses, and evaluation results.

2.3 Functional Requirements

ID	Requirement
FR-01	Start a new interview session.
FR-02	Select interview domain and difficulty level.
FR-03	Generate AI-based interview questions.
FR-04	Submit interview responses.
FR-05	Evaluate responses and generate feedback.
FR-06	Store interview history and results.
FR-07	Display previous interview sessions.

2.4 Non-Functional Requirements

Category	Requirement
Performance	Fast response for interview requests.
Security	Validate user input and secure API communication.
Reliability	Ensure stable communication between all components.
Scalability	Support future feature expansion.
Usability	Provide an intuitive and responsive interface.

2.5 Software Requirements

Software	Purpose
Python 3.12+	Backend development
React.js	Frontend development
FastAPI	REST API framework
LangChain	AI workflow management
Google Gemini	Conversational AI model

SQLite	Database
Git	Version control
Postman	API testing

2.6 Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i3 or higher
RAM	8 GB
Storage	10 GB free space
Internet	Stable broadband connection

2.7 System Workflow

Figure 2.1: Overall System Workflow

User → React Frontend → FastAPI Backend → Google Gemini → SQLite Database → User

2.8 Assumptions

- Users have a stable internet connection.
- The Google Gemini API is available during interview sessions.
- The application is accessed through a modern web browser.
- Required software components are correctly configured before deployment.

2.9 Constraints

- The current version supports only text-based interviews.
- AI response quality depends on the external LLM service.
- SQLite is intended for small to medium-sized deployments.
- Internet connectivity is required for AI-generated responses.

Chapter 3 – Software Architecture

3.1 Overview

The Technical Interview Coach follows a modular three-tier architecture consisting of the presentation layer, application layer, and data layer. This architecture separates the user interface, business logic, AI processing, and data storage, making the application easier to develop, maintain, and scale.

3.2 Architectural Style

The application adopts a **Client–Server Architecture** with RESTful communication. The React frontend sends user requests to the FastAPI backend, which processes the requests, interacts with Google Gemini for AI-generated responses, and stores interview data in the SQLite database.

Figure 3.1: High-Level Architecture

React Frontend → FastAPI Backend → Google Gemini & SQLite Database

3.3 Architecture Layers

Layer	Responsibility
Presentation Layer	Provides the user interface and captures user interactions.
Application Layer	Processes requests, manages interview sessions, and communicates with AI services.
AI Layer	Generates interview questions and evaluates candidate responses.

Data Layer	Stores interview sessions, responses, scores, and feedback.
------------	---

3.4 System Modules

Module	Description
Authentication (<i>Future</i>)	Manages user login and access control.
Interview Management	Creates and manages interview sessions.
AI Evaluation	Generates questions and analyses responses using Google Gemini.
Result Management	Calculates scores and generates interview feedback.
History Management	Stores and retrieves previous interview sessions.
Database Module	Performs database operations using SQLite.

3.5 Module Interaction

The modules communicate through REST APIs. The frontend initiates interview requests, the backend processes business logic, the AI module generates responses, and the database stores interview information. This modular approach reduces dependencies and simplifies future enhancements.

3.6 Design Principles

The system architecture follows these software engineering principles:

- Modular design for easier maintenance.
- Separation of concerns between frontend, backend, AI, and database.
- RESTful communication using JSON.
- Reusable and scalable components.

- Loose coupling between application modules.

3.7 Architecture Advantages

- Supports independent frontend and backend development.
- Simplifies testing and debugging.
- Enables easy integration of new AI models.
- Supports future migration to cloud-based databases.
- Improves maintainability and scalability.

Chapter 4 – Technology Stack

4.1 Overview

The Technical Interview Coach is developed using modern web technologies and AI frameworks that provide high performance, scalability, and maintainability. Each technology is selected based on its reliability, ease of integration, and suitability for building AI-powered web applications.

4.2 Technology Stack

Layer	Technology	Purpose
Frontend	React.js	Builds the user interface
Styling	Tailwind CSS	Creates responsive UI components
Backend	FastAPI	Develops RESTful APIs
Programming Language	Python 3.12+	Implements backend logic
AI Model	Google Gemini	Generates interview questions and feedback

AI Framework	LangChain	Manages AI prompts and workflows
Database	SQLite	Stores interview data
API Testing	Postman	Tests REST API endpoints
Version Control	Git & GitHub	Source code management

4.3 Technology Integration

The application integrates multiple technologies to provide a seamless interview experience. React handles the user interface, FastAPI manages backend operations, LangChain communicates with Google Gemini for AI processing, and SQLite stores interview records. All components interact through REST APIs using JSON data.

Figure 4.1: Technology Integration

React.js → FastAPI → LangChain → Google Gemini → SQLite

4.4 Development Tools

Tool	Purpose
Visual Studio Code	Source code development
GitHub	Version control and collaboration
Postman	API testing and validation
Git	Source code tracking

Chrome Developer Tools UI debugging and inspection

4.5 Key Libraries

Library	Function
FastAPI	REST API development
SQLAlchemy	Database operations
Pydantic	Request and response validation
LangChain	AI workflow management
Google Generative AI SDK	Gemini integration
Axios	Frontend API communication

4.6 Benefits of the Technology Stack

- Modern and lightweight architecture.
- Fast API performance with asynchronous processing.
- AI-powered interview generation using Google Gemini.
- Responsive user interface with React.
- Easy maintenance through modular development.
- Supports future migration to cloud-based deployment.

Chapter 5 – Database Design

5.1 Overview

The Technical Interview Coach uses **SQLite** as its relational database to manage application data. The database stores user information, interview sessions, candidate responses, and AI-generated evaluation results. The schema is designed to minimise redundancy while ensuring efficient data retrieval and easy maintenance.

5.2 Database Tables

Table Name	Purpose
Users	Stores user profile information.
Interview_Sessions	Maintains interview session details.
Interview_Response s	Stores questions and candidate answers.
Interview_Results	Stores scores, feedback, and interview summaries.

5.3 Table Structure

Users

Field	Data Type	Description
user_id	Integer (PK)	Unique user identifier
name	Varchar	User name
email	Varchar	Email address

Interview_Sessions

Field	Data Type	Description
session_id	Integer (PK)	Unique session identifier
user_id	Integer (FK)	References Users table
domain	Varchar	Selected interview domain
difficulty	Varchar	Interview difficulty level
interview_date	DateTime	Session creation time

Interview_Responses

Field	Data Type	Description
response_id	Integer (PK)	Unique response identifier
session_id	Integer (FK)	References Interview_Sessions
question	Text	AI-generated interview question
answer	Text	Candidate response

Interview_Results

Field	Data Type	Description
-------	-----------	-------------

result_id	Integer (PK)	Unique result identifier
session_id	Integer (FK)	References Interview_Sessions
score	Integer	Interview score
feedback	Text	AI-generated feedback

5.4 Entity Relationship

Figure 5.1: Database Relationship

Users → Interview Sessions → Interview Responses → Interview Results

5.5 Database Operations

Operation	Description
Create	Create a new interview session.
Read	Retrieve interview history and results.
Update	Update interview responses during a session.
Delete	Remove obsolete interview records if required.

5.6 Database Features

- Relational database design.
- Primary and foreign key relationships.

- Efficient storage of interview records.
- Reduced data redundancy through normalization.
- Supports future migration to PostgreSQL or MySQL.

Chapter 6 – API Design

6.1 Overview

The Technical Interview Coach uses RESTful APIs to enable communication between the React frontend and the FastAPI backend. APIs receive user requests, process interview sessions, interact with the Google Gemini model, and return responses in JSON format. This architecture ensures efficient, secure, and scalable communication.

6.2 API Endpoints

Method	Endpoint	Description
POST	<code>/api/interview/start</code>	Starts a new interview session
POST	<code>/api/interview/question</code>	Generates the next interview question
POST	<code>/api/interview/answer</code>	Submits the candidate's response
GET	<code>/api/interview/result/{id}</code>	Retrieves interview results
GET	<code>/api/interview/history</code>	Displays interview history
GET	<code>/api/health</code>	Checks application status

6.3 API Request Flow

Figure 6.1: API Communication

User → React Frontend → FastAPI → Google Gemini / SQLite → JSON
Response → User

6.4 Request and Response Format

All APIs use the **JSON** format for data exchange between the frontend and backend.

Request	Response
Interview domain	Session ID
Difficulty level	Interview question
Candidate answer	AI feedback
Session ID	Score and result

6.5 HTTP Status Codes

Status Code	Meaning
200	Request processed successfully
201	Resource created successfully
400	Invalid request

401 Unauthorized access

404 Resource not found

500 Internal server error

6.6 API Security

Feature	Description
Input Validation	Validates request data using Pydantic
HTTPS	Secures data transmission
Environment Variables	Protects API keys and configuration
Error Handling	Returns standardised error responses

6.7 API Features

- RESTful architecture.
- JSON-based communication.
- Stateless request handling.
- Fast response processing.
- Easy integration with external services.
- Modular API design for future expansion.

Chapter 7 – User Interface Design

7.1 Overview

The Technical Interview Coach provides a simple, responsive, and user-friendly interface that enables users to participate in AI-driven interview sessions with minimal effort. The interface is designed using React.js and Tailwind CSS to ensure consistency across desktop and mobile devices while maintaining a clean and intuitive user experience.

7.2 User Interface Screens

Screen	Purpose
Home Page	Displays application information and allows users to start an interview.
Interview Setup	Enables users to select the interview domain and difficulty level.
Interview Session	Displays AI-generated questions and accepts user responses.
Result Page	Shows interview score, feedback, and improvement suggestions.
History Page	Displays previous interview sessions and results.

7.3 Navigation Flow

Figure 7.1: User Navigation



7.4 User Interface Components

Component	Description
Navigation Bar	Provides access to different application pages.

Input Forms	Collect interview preferences and user responses.
Question Panel	Displays AI-generated interview questions.
Response Box	Allows users to submit answers.
Result Dashboard	Presents scores, feedback, and recommendations.

7.5 UI Design Principles

Principle	Description
Simplicity	Easy-to-use interface with minimal distractions.
Consistency	Uniform colours, fonts, and navigation across all screens.
Responsiveness	Supports desktop, tablet, and mobile devices.
Accessibility	Clear labels and readable content for improved usability.
Performance	Fast page loading and smooth user interaction.

7.6 User Experience Features

- Clean and responsive interface.
- Simple navigation between application screens.
- Real-time display of AI-generated interview questions.

- Instant feedback after interview completion.
- Mobile-friendly design with consistent layout.
- Easy access to interview history and previous results.

Chapter 8 – Testing Strategy

8.1 Overview

Testing is an essential phase of software development that ensures the Technical Interview Coach functions correctly, reliably, and efficiently. The application is tested to verify the frontend, backend, APIs, AI integration, and database operations before deployment.

8.2 Testing Objectives

- Verify that all functional requirements are implemented correctly.
- Ensure seamless communication between system components.
- Validate AI-generated responses and interview workflow.
- Identify and resolve software defects before deployment.
- Improve overall system reliability and performance.

8.3 Testing Types

Testing Type	Purpose
Unit Testing	Tests individual modules and functions independently.
Integration Testing	Verifies communication between frontend, backend, AI, and database.
System Testing	Validates the complete application workflow.
Functional Testing	Ensures application features work as expected.
User Acceptance Testing (UAT)	Confirms the system meets user requirements.

8.4 Test Scenarios

Test ID	Test Scenario	Expected Result
TS-01	Start interview session	Session created successfully
TS-02	Select domain and difficulty	Selection saved correctly
TS-03	Submit interview response	AI generates evaluation and feedback
TS-04	View interview history	Previous interview records displayed
TS-05	Invalid API request	Appropriate error message returned

8.5 Testing Tools

Tool	Purpose
Pytest	Backend unit testing
Postman	REST API testing
React Testing Library	Frontend component testing
Chrome Developer Tools	UI testing and debugging
GitHub	Version control and code review

8.6 Test Execution Process

Figure 8.1: Testing Workflow

Unit Testing → Integration Testing → System Testing → User Acceptance Testing → Production Deployment

8.7 Expected Outcome

The testing process confirms that the Technical Interview Coach performs as expected under normal operating conditions. All major functionalities, including interview generation, AI evaluation, API communication, and database operations, are validated to ensure a stable, secure, and reliable user experience.

Chapter 9 – Deployment and Security

9.1 Overview

The Technical Interview Coach is deployed as a web-based application using a client-server architecture. The React frontend communicates with the FastAPI backend through REST APIs, while the backend integrates with Google Gemini for AI processing and SQLite for data storage. The deployment is designed to provide secure, reliable, and efficient application access.

9.2 Deployment Environment

Component	Technology
Frontend	React.js
Backend	FastAPI
AI Service	Google Gemini API
Database	SQLite

Version Control GitHub

API Communication REST API (HTTPS)

9.3 Deployment Workflow

Figure 9.1: Deployment Process

Development → Testing → GitHub Repository → Production Deployment → End Users

9.4 Security Features

Security Measure	Purpose
HTTPS	Encrypts data transmitted between client and server.
Input Validation	Prevents invalid or malicious user input.
Environment Variables	Protects API keys and configuration settings.
Error Handling	Prevents exposure of sensitive system information.
Data Validation	Ensures only valid records are stored in the database.

9.5 Deployment Steps

1. Develop and test the application locally.
2. Store the source code in the GitHub repository.
3. Configure the backend and database.
4. Set the required environment variables.
5. Deploy the frontend and backend.
6. Perform final testing before release.

9.6 Deployment Benefits

- Simple deployment architecture.
- Secure communication using HTTPS.
- Easy maintenance and updates.
- Modular design for future cloud migration.
- Supports scalable application development.

9.7 Future Deployment Scope

The application can be enhanced by deploying it on cloud platforms such as AWS, Microsoft Azure, or Google Cloud. Future versions may also replace SQLite with PostgreSQL or MySQL to support higher user traffic and improve scalability.

Chapter 10 – Future Enhancements and Conclusion

10.1 Future Enhancements

The Technical Interview Coach has been designed with a modular architecture that allows new features to be integrated with minimal changes to the existing system. Future improvements will enhance the application's capabilities, provide a better user experience, and support enterprise-level deployment.

Enhancement	Description
Voice-Based Interviews	Enable AI-powered voice interaction using speech recognition and text-to-speech.
Coding Assessment	Integrate an online code editor with automated code evaluation.
Resume-Based Interviews	Generate interview questions based on the candidate's uploaded resume.

Multi-Language Support	Support interview sessions in multiple languages.
User Authentication	Implement secure login using JWT or OAuth 2.0.
Performance Dashboard	Display interview history, scores, and progress through visual reports.
Cloud Database	Replace SQLite with PostgreSQL or MySQL for improved scalability.
Cloud Deployment	Deploy the application on AWS, Azure, or Google Cloud Platform.

10.2 Project Achievements

The Technical Interview Coach successfully achieves the primary objectives defined during the project planning phase.

- Developed an AI-powered technical interview platform.
- Generated dynamic interview questions using Google Gemini.
- Evaluated candidate responses and provided personalised feedback.
- Maintained interview history using a relational database.
- Built a responsive web interface using React.js.
- Implemented RESTful APIs with FastAPI.
- Designed a modular architecture for future enhancements.

10.3 Conclusion

The **Technical Interview Coach** demonstrates the practical application of Conversational AI in technical interview preparation. By integrating React.js, FastAPI, LangChain, Google Gemini, and SQLite, the system delivers an interactive platform that simulates real interview experiences and provides meaningful AI-generated feedback.

The modular architecture, REST-based communication, and scalable design ensure that the application is easy to maintain, extend, and deploy. The proposed future enhancements provide a clear roadmap for expanding the platform with advanced features such as voice interviews, coding assessments, cloud deployment, and performance analytics.

Overall, the project meets its functional objectives while showcasing the effective use of modern AI technologies to improve technical interview preparation and learning outcomes.

References

1. Google. *Gemini API Documentation*. <https://ai.google.dev/>
2. FastAPI. *FastAPI Documentation*. <https://fastapi.tiangolo.com/>
3. React. *React Documentation*. <https://react.dev/>
4. LangChain. *LangChain Documentation*. <https://python.langchain.com/>
5. SQLite. *SQLite Documentation*. <https://www.sqlite.org/>
6. Python Software Foundation. *Python Documentation*. <https://docs.python.org/>
7. GitHub. *GitHub Documentation*. <https://docs.github.com/>